

INDUCTION AND RECURSION

Mathematical induction

Mathematical Induction is a technique or method to prove mathematical statement or formula or theorem which is thought to be true for each and every natural number n . By generalizing this in the form of a principle which we would use to prove any mathematical statement is principle of mathematical induction.

Principle of Mathematical Induction

Let $P(n)$ be a property that is defined for integers n , and let a be a fixed integer.

Suppose the following two statements are true:

1. $P(a)$ is true.
2. For all integers $k \geq a$, if $P(k)$ is true then $P(k + 1)$ is true.

Then the statement for all integers $n \geq a$, $P(n)$ is true.

Proving a statement by mathematical induction is a two-step process. The first step is called the basis step, and the second step is called the inductive step.

Method of Proof by Mathematical Induction

Consider a statement of the form, "For all integers $n \geq a$, a property $P(n)$ is true."

To prove such a statement, perform the following two steps:

Step 1: **Basis step:** Show that $P(a)$ is true.

Step 2: **Inductive step:** Show that for all integers $k \geq a$, if $P(k)$ is true then $P(k + 1)$ is true.

$P(k)$ To perform step 2, suppose that $P(k)$ is true, where k is any particular but arbitrarily chosen integer with $k \geq a$. Then show that $P(k + 1)$ is true. [This is called the inductive hypothesis.]

Strong mathematical induction

Strong mathematical induction is similar to mathematical induction in that it is a technique for establishing the truth of a sequence of statements about integers. Also, a proof by strong mathematical induction consists of a basis step and an inductive step. However, the basis step may contain proofs for several initial values, and in the inductive step the truth of the predicate $P(n)$ is assumed not just for one value of n but for all values through k , and then the truth of $P(k + 1)$ is proved.

Principle of Strong Mathematical Induction

Let $P(n)$ be a property that is defined for integers n , and let a and b be fixed integers with $a \leq b$. Suppose the following two statements are true:

1. **Basis step:** $P(a)$, $P(a + 1)$, ..., and $P(b)$ are all true.
2. **Inductive step:** For any integer $k \geq b$, if $P(i)$ is true for all integers i from a through k , then $P(k + 1)$ is true.

Then the statement for all integers $n \geq a$, $P(n)$ is true. (The supposition that $P(i)$ is true for all integers from a through k is called the inductive hypothesis. Another way to state inductive hypothesis is to say that $P(a)$, $P(a + 1)$, ..., $P(k)$ are all true.)

The Well Ordering Principle

Well Ordering Principle states that a least element exists in any non empty set of positive integers. Every non-empty set S of positive integers contains a least element; that is, there is some element a of S such that $a \leq b$ for all elements b of S . The well-ordering property can often be used directly in proofs.

If S is a set of integers, each element in S is greater than or equal to 0, and S contains at least one element, then of course one of the elements has to be the smallest.

Example: Show that if n is an integer greater than 1, then n can be written as the product of primes.

Proof: Let $P(n)$ be the proposition that n can be written as the product of primes.

Basis Step : $P(2)$ is true, because 2 can be written as the product of one prime, itself.

(note that $n = 2$ is the first case (first prime number) we need to establish.)

Inductive Step: The inductive hypothesis is the assumption that $P(i)$ is true for all integers i with $2 \leq i \leq k$ i.e., the assumption that i can be written as the product of primes whenever j is a positive integer at least 2 and not exceeding k .

To complete the inductive step, it must be shown that $P(k + 1)$ is true under this assumption, that is, that $k + 1$ is the product of primes.

There are two cases to consider, namely, when $k + 1$ is prime and when $k + 1$ is composite.

If $k + 1$ is prime, we immediately see that $(k + 1)$ is true.

Otherwise, $k + 1$ is composite and $k + 1$ can be written as the product of two positive integers a and b with $2 \leq a \leq b < k + 1$. Because both a and b are integers at least 2 and not exceeding k , we can use the inductive hypothesis to write both a and b as the product of primes. Thus, if $k + 1$ is composite, it can be written as the product of primes, namely, those primes in the factorization of a and those in the factorization of b .

Example:

The following calculations indicate that it is possible to write (without regard to order) the integer 14, 15, 16 using only 3's and/or 8's as summands:

$$14 = 3 + 3 + 8$$

$$15 = 3 + 3 + 3 + 3 + 3$$

$$16 = 8 + 8$$

On the basis of these three results, we make the conjecture

For every $n \in \mathbb{Z}'$ where $n \geq 14$

$P(n)$ n can be written as a sum of 3's and/or 8's.

Proof: It is apparent that the statements $P(14)$ $P(15)$ and $P(16)$ are true - and this establishes our basis step. ($n_0 = 14$ and $n_1 = 16$)

For the inductive step we assume the truth of the statements

$P(14)$, $P(15)$, ..., $P(k - 2)$, $P(k - 1)$ and $P(k)$ for some $k \in \mathbb{Z}$, where $k \geq 16$.

[The assumption of the truth of these $(k - 14) + 1$ statements constitutes our induction hypothesis.]

And now $n = k + 1$ then $n \geq 17$ and $k + 1 = (k - 2) + 3$.

But since $14 \leq k - 2 \leq k$ from the truth of $P(k - 2)$ we know that $P(k - 2)$ can be written as a sum of 3's and/or 8's; so $(k + 1) = (k - 2) + 3$ can also be written in this form.

Consequently, $P(n)$ is true for all $n \geq 14$ by alternative form of the principle of Mathematical Induction.

Example:

Let us consider the integer sequence $a_0, a_1, a_2, a_3 \dots$ where

$$a_0=1, a_1=2, a_2=3,$$

$$a_n=a_{n-1}+a_{n-2}+a_{n-3} \text{ and for all } n \text{ in } \mathbb{Z}^+ \text{ where } n \geq 3$$

(Then, for instance, we find that $a_3 = a_2 + a_1 + a_0 = 3 + 2 + 1 = 6$

$$a_4 = a_3 + a_2 + a_1 = 6 + 3 + 2 = 11; \text{ and } a_5 = a_4 + a_3 + a_2 = 11 + 6 + 3 = 20)$$

We claim that the entries in this sequence are such that $a_n \leq 3^n$ for all $n \in \mathbb{N}$ - that is, $\forall n \in \mathbb{N} P'(n)$ where $P'(n)$ is the open statement: $a_n \leq 3^n$

For the basis step, we observe that

$$(i) \quad a_0=1=3^0 \leq 3^0$$

$$(ii) \quad a_1=2 \leq 3=3^1$$

$$(iii) \quad a_2=3 \leq 9=3^2$$

Consequently, we know that $P'(1)$ and $P'(2)$ are true statements,

So now we turn attention to the inductive step where we assume the truth of the statement

$P'(0), P'(1), P'(2), \dots, P'(k-1), P'(k)$ for some k in $\mathbb{Z}^+$ where $k \geq 2$ For the case where $n = k + 1 \geq 3$

we see that

$$\begin{aligned} a_{k+1} &= a_k + a_{k-1} + a_{k-2} \\ &\leq 3^k + 3^{k-1} + 3^{k-2} \\ &\leq 3^k + 3^k + 3^k = 3(3^k) = 3^{k+1} \end{aligned}$$

So $[P'(k-2) \wedge P'(k-1) \wedge P'(k)] \Rightarrow P'(k+1)$

Therefore it follows from the alternative form of the Principle of Mathematical Induction that $a_n \leq 3^n$ for all $n \in \mathbb{N}$.

Recursively define functions

We use two steps to define a function with the set of nonnegative integers as its domain:

Basis Step: Specify the value of the function at zero.

Recursive Step: Give a rule for finding its value at an integer from its values at smaller integers such a definition is called a recursive or inductive definition.

Example:

Suppose that f is defined recursively by $f(0) = 3$ $f(n+1) = 2f(n) + 3$

Find $f(1)$, $f(2)$, $f(3)$, and $f(4)$

Solution:

From the recursive definition it follows that

$$n = 0 \text{ then } f(1) = 2f(0) + 3 = 2 * 3 + 3 = 9$$

$$n = 1 \text{ then } f(2) = 2f(1) + 3 = 2 * 9 + 3 = 21$$

$$n = 2 \text{ then } f(3) = 2f(2) + 3 = 2 * 21 + 3 = 45$$

$$\text{If } n = 3 \text{ then } f(4) = 2f(3) + 3 = 2 * 45 + 3 = 93$$

Recursively defined functions are well defined. That is, for every positive integer, the value of the function at this integer is determined in an unambiguous way.

Recursively defined sets and functions:

Recursive definitions of sets have two parts, a basis step and a recursive step. In the basis step, an initial collection of elements is specified. In the recursive step, rules for forming new elements in the set from those already known to be in the set are provided. Recursive definitions may also include an exclusion rule, which specifies that a recursively defined set contains nothing other than those elements specified in the basis step or generated by applications of the recursive step. Assume S is a set. We use two steps to define the elements of S .

1. Basis Step: Specify an initial collection of elements.
2. Recursive Step: Give a rule for forming new elements from those already known to be in S .

Structural Induction

A proof by structural induction consists of two parts.

1. **Basis Step:** Show that the result holds for all elements specified in the basis step of the recursive definition to be in the set.
2. **Inductive Step:** Show that if the statement is true for each of the elements used to construct new elements in the recursive step of the definition, the result holds for these new elements

Structural induction is a technique that allows us to apply induction or recursive definition even if there is no integer. Structural Induction are similar to regular induction, but can make proofs much easier.

Example: Let T be a full binary tree over an alphabet A . Then we have $n(T) \leq 2^{h(T)+1} - 1$.

Proof: By structural induction.

Basis step: For r in A , we have $n(r)=1$ and $h(r)=0$, therefore, we have $n(r) = 1 \leq 2^{(0+1)} - 1 = 2^{h(r)+1} - 1$, as claimed.

Inductive step: Suppose that L and R are full binary trees that satisfy $n(L) \leq 2^{h(L)+1} - 1$ and $n(R) \leq 2^{h(R)+1} - 1$.

Then the tree T satisfies:

$$n(T) = 1 + n(L) + n(R) \leq 1 + 2^{h(L)+1} - 1 + 2^{h(R)+1} - 1$$

$$\leq 2 \max(2^{h(L)+1}, 2^{h(R)+1}) - 1,$$

$$\text{since } a+b \leq 2\max(a,b) \leq 2 \cdot 2^{\max(h(L),h(R))+1} - 1 = 2^{h(T)+1} - 1 = 2^{h(T)+1} - 1.$$

Recursive algorithm

An algorithm is called recursive if it solves a problems by reducing it to an instance of thllhe same problems with smaller input.

Example: Give a recursive algorithm for computing $n!$, where n is a nonnegative integer.

Solution:

We can build a recursive algorithm that finds $n!$, where n is a nonnegative integer, based on the recursive definition of $n!$,

Which specifies that $n! = n \cdot (n - 1)!$ When n is a positive integer, and that $0! = 1$

i.e. we trace the steps used by the algorithm to compute $4!$.

First, we use the recursive step to write $4! = 4 \cdot 3!$

We then use the recursive step repeatedly to write $3! = 3 \cdot 2!$

$2! = 2 \cdot 1!$ and

$1! = 1 \cdot 0!$ Inserting the value of $0! = 1$ and

Working back through the steps, we see that

$$2! = 2 \cdot 1! = 2$$

$$3! = 3 \cdot 2! = 3 \cdot 2 = 6 \text{ and}$$

$$4! = 4 \cdot 3! = 4 \cdot 6 = 24$$

Algorithm

A Recursive Algorithm for Computing $n!$.

Procedure: factorial (n : nonnegative integer)

 if $n = 0$ then return 1

 else return $n \cdot \text{factorial}(n - 1)$

 {output is $n!$ }

Example 2:

Give a recursive algorithm for computing the greatest common divisor of two nonnegative integers a and b with $a < b$.

Solution:

We can base a recursive algorithm on the reduction

$\text{gcd}(a, b) = \text{gcd}(b \bmod a, a)$ and the condition $\text{gcd}(0, b) = b$ when $b > 0$

We illustrate the workings of Algorithm with a trace when the input is $a = 5$ and $b = 8$

With this input, the algorithm uses the "else" clause to find that

$\text{gcd}(5, 8) = \text{gcd}(8 \bmod 5, 5) = \text{gcd}(3, 5)$.

It uses this clause again to find that $\text{gcd}(3, 5) = \text{gcd}(5 \bmod 3, 3) = \text{gcd}(2, 3)$

then to get $\text{gcd}(2, 3) = \text{gcd}(3 \bmod 2, 2) = \text{gcd}(1, 2)$,

then to get $\text{gcd}(1, 2) = \text{gcd}(2 \bmod 1, 1) = \text{gcd}(0, 1)$.

Finally, to find $\text{gcd}(0, 1)$ it uses the first step with $a = 0$ to find that $\text{gcd}(0, 1) = 1$.

Consequently, the algorithm finds that $\text{gcd}(5, 8) = 1$.

Algorithm: A Recursive Algorithm for Computing $\text{gcd}(a, b)$.

Procedure: $\text{gcd}(a, b)$: nonnegative integers with $a < b$

 if $a = 0$ then return b

 else return $\text{gcd}(b \bmod a, a)$

 {output is $\text{gcd}(a, b)$ }

Some algorithms:

Algorithm: An Iterative Algorithm for Computing Fibonacci Numbers

Procedure: iterative fibonacci (n : nonnegative integer)

 if $n = 0$ then return 0

 else

$x := 0$

$y := 1$

 for $i := 1$ to $n - 1$

$z := x + y$

$x := y$

$y := z$

 return y

 {output is the n^{th} Fibonacci number}

Algorithm: A Recursive Merge Sort.

Procedure: mergesort ($L = a_1, \dots, a_n$)

 if $n > 1$ then

$m := \lfloor n / 2 \rfloor$

$L_1 = a_1, a_2, \dots, a_m$

$L_2 = a_{m+1}, a_{m+2}, \dots, a_n$

 L: merge (mergesort (L_1), mergesort (L_2))

 {L is now sorted into elements in nondecreasing order}

Algorithm: Merging Two Lists.

Procedure: merge (L_1, L_2 : sorted lists)

 L := empty list

 while L_1 and L_2 are both nonempty

 remove smaller of first elements from L_1 , and L_2 from its list; put it at the right end of L if this removal makes one list empty then remove all elements from the other list and append them to L

return L {L is the merged list with elements in increasing order}

Types of Recursion

1. Direct Recursion

In direct recursion, a function calls itself directly.

This is the most common type of recursion and is used in many classic examples like factorial calculation, Fibonacci sequence, and binary search.

2. Indirect Recursion

In indirect recursion, a function does not call itself directly.

Instead, it calls another function, which eventually calls the original function. This creates a cycle of calls between multiple functions.

3. Tail Recursion

In tail recursion, the recursive call is the last operation in the function.

There are no further calculations or operations after the recursive call. This type of recursion can be optimized by the compiler to avoid adding new frames to the call stack, making it more memory-efficient.

4. Non-Tail Recursion

In non-tail recursion, the recursive call is not the last operation in the function.

After the recursive call, there are additional operations that need to be performed. This type of recursion is not as memory-efficient as tail recursion because it requires keeping track of additional operations for each recursive call.

Program corrections

Mathematical induction, and its variant strong induction, can be used to prove that a recursive algorithm is correct, that is, that it produces the desired output for all possible input values.

Example:

Prove that following Algorithm: A Recursive Algorithm for Computing a^n .

Procedure: power(a: nonzero real number, n: nonnegative integer)

 if $n = 0$ then return 1

 else return a power(a, $n - 1$)

 {output is a^n }

Which computes powers of real numbers, is correct.

Solution:

We use mathematical induction on the exponent n .

Basis Step: If $n = 0$ the first step of the algorithm tells us that $\text{power}(a, 0) = 1$.

This is correct because $a^0 = 1$ for every nonzero real number a .

This completes the basis step.

Inductive Step: The inductive hypothesis is the statement that $\text{power}(a, k) = a^k$ for all $a \neq 0$ for an arbitrary nonnegative integer k .

That is, the inductive hypothesis is the statement that the algorithm correctly computes a^k .

To complete the inductive step, we show that if the inductive hypothesis is true, then the algorithm correctly computes a^{k+1} .

Because $k + 1$ is a positive integer, when the algorithm computes a^{k+1} , the algorithm sets $\text{power}(a, k+1) = a \cdot \text{power}(a, k)$.

By the inductive hypothesis, we have

$\text{power}(a, k) = a^k$ so $\text{power}(a, k + 1) = a \cdot \text{power}(a, k) = a \cdot a^k = a^{k+1}$.

This completes the inductive step.

We have completed the basis step and the inductive step, so we can conclude that above Algorithm always computes a correctly when $a \neq 0$ and n is a nonnegative integer.

Program Verification

We need to prove that the given program always gives the correct output. Proving the program correctness is called as program verification.

Program verification uses rules of inference, proof techniques and mathematical induction.

A program is correct if it produces the correct output for every possible input.

A program has partial correctness if it produces the correct output for every input for which the program eventually halts.

Therefore, a program is correct if and only if it has partial correctness and terminates.

A program's input and output specification can be given using initial and final assertions.

- Initial Assertion (p): The initial assertion p is the condition that the program's input (its initial state) is guaranteed (by its user) to satisfy. It gives the properties that the input values must have.

- Final Assertion (q): The final assertion q is the condition that the output produced by the program (its final state) is required to satisfy. It gives the properties that the output of the program should have.

The appropriate initial and final assertions must be provided when a program is checked.

Definition: A program, or program segment, S is said to be partially correct with respect to the initial assertion p and the final assertion q if whenever p is true for the input values of S and S terminates, then q is true for the output values of S . The notation $p\{S\}q$ indicates that the program, or program segment, S is partially correct with respect to the initial assertion p and the final assertion q .

Example:

Show that the program segment

$y := 8$

$z := x + y$

is correct with respect to the initial assertion $p: x = 1$ and the final assertion $q: z = 9$

Solution:

Suppose that p is true, so that $x = 1$ as the program begins.

Then y is assigned the value 8, and

z is assigned the sum of the values of x and y , which is 9.

Hence, S is correct with respect to the initial assertion p and the final assertion q .

Thus, $p\{S\}q$ is true.

RELATIONS

Cartesian products of sets:

Given two non-empty sets A and B, the set of all ordered pairs (x, y), where $x \in A$ and $y \in B$ is called Cartesian product of A and B; symbolically, we write

$$A \times B = \{(x, y) \mid x \in A \text{ and } y \in B\}$$

If $A = \{1, 2, 3\}$ and $B = \{4, 5\}$, then

$$A \times B = \{(1, 4), (2, 4), (3, 4), (1, 5), (2, 5), (3, 5)\} \text{ and}$$

$$B \times A = \{(4, 1), (4, 2), (4, 3), (5, 1), (5, 2), (5, 3)\}$$

Relation:

A Relation R from a non-empty set A to a non-empty set B is a subset of the Cartesian product set $A \times B$. The subset is derived by describing a relationship between the first element and the second element of the ordered pairs in $A \times B$.

Domain and range of relation:

The set of all first elements in a relation R, is called the domain of the relation R, and the set of all second elements called images, is called the range of R.

$$\text{Domain} = \{a \in A \mid (a, b) \in R\}$$

$$\text{Range} = \{b \in B \mid (a, b) \in R\}$$

Kinds of relations:

One to One Relation: The relation R from the set A to the set B is said to be one to one relation, i.e., $(a, b) \in R$, $(c, d) \in R$, then $b = d \Rightarrow a = c$.

Many to One Relation: The relation R from the set A to the set B is said to be many to one relation, i.e., $(a, b) \in R$ and $(c, b) \in R$, for some $a, c \in A$ and $b \in B$ with $a \neq c$.

One to Many Relation: The relation R from the set A to the set B is said to be one to many relation, i.e., $(a, b) \in R$, $(a, c) \in R$, for some $a \in A$ and $b, c \in B$ with $b \neq c$.

Many to Many Relation: The relation R from the set A to the set B is said to be many to many relation, i.e., $(a, b) \in R$, $(a, c) \in R$ and $(d, b) \in R$, $(d, c) \in R$ for some $a, d \in A$ and $b, c \in B$ with $a \neq d$ and $b \neq c$.

Properties of relation:

Reflexive Relation:

A relation R on a set A is called reflexive if $(a, a) \in R$ for every element $a \in A$.

Example: Let the set $A = \{1, 2, 3\}$ and the relation R be $R = \{(1, 1), (2, 2), (3, 3)\}$.

Anti-reflexive or Irreflexive Relation:

A relation 'R' on a set A is called irreflexive if for every $a \in A$, $(a, a) \notin R$.

Example: Let the set $A = \{1, 2, 3\}$ and the relation $R = \{(1, 2), (2, 3)\}$. This relation is irreflexive because no pair (a, a) is in R.

Symmetric Relation:

A relation R on a set A is called symmetric if $(b, a) \in R$ whenever $(a, b) \in R$ for all $a, b \in A$ i.e., $(a, b) \in R \Rightarrow (b, a) \in R$ for all $a, b \in A$.

Example: Let the set $A = \{1, 2, 3\}$ and the relation $R = \{(1, 2), (2, 1), (3, 3)\}$.

Anti-symmetric relation:

A relation R on a set A such that for all $a, b \in A$ if $(a, b) \in R$ and $(b, a) \in R$ then $a = b$ is called Anti-symmetric.

i.e., if $(a, b) \in R$ and $(b, a) \in R \Rightarrow a = b$, for all $a, b \in A$.

Example: Let the set $A = \{1, 2, 3\}$ and the relation $R = \{(1, 2), (2, 1), (3, 3)\}$.

This relation is **not antisymmetric** because $(1, 2)$ and $(2, 1)$ are in R, but $1 \neq 2$.

However, if the relation was $R=\{(1,2),(3,3)\}$, it would be antisymmetric since no pair (a, b) and (b, a) exists with $a \neq b$.

Asymmetric Relation:

A relation R on set A is said to be asymmetric, if $(a,b) \in R$, then $(b,a) \notin R$.

i.e, if $(a, b) \in R \Rightarrow (b, a) \notin R$, for all $a, b \in A$

Example: Let the set $A=\{1,2,3\}$ and the relation $R=\{(1,2),(2,3)\}$.

This relation is asymmetric because if $(1, 2)$ is in R , $(2, 1)$ is not in R , and similarly for $(2, 3)$.

Transitive Relation:

A relation R on a set A is called transitive if whenever $(a, b) \in R$ and $(b, c) \in R$ then (a, c) for all $a, b, c \in A$.

i.e., if $(a, b) \in R$ and $(b, c) \in R \Rightarrow (a, c) \in R$, for all $a, b, c \in A$.

Example: Let the set $A=\{1,2,3\}$ and the relation $R=\{(1,2),(2,3),(1,3)\}$.

This relation is transitive because, since $(1, 2)$ and $(2, 3)$ are in R , the pair $(1, 3)$ is also in R .

Universal Relation:

The relation $R = A \times A$ on A is called universal relation in A .

Example: $A = \{1, 2, 3\}$

$R = A \times A = \{(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2), (3, 3)\}$

Inverse Relation (R^{-1}):

Let R be a relation from the set A into B . The inverse relation denoted by R^{-1} from B to A and it's defined by $R^{-1} = \{(b, a) / (a, b) \in R \text{ for all } a, b\}$

$R^{-1} = \{(b, a) / (a, b) \in R \text{ for all } a, b\}$

Example: Let the set $A=\{1,2,3\}$ and the relation $R=\{(1,2),(2,3),(1,3)\}$.

$R^{-1} = \{(2,1),(3,2),(3,1)\}$.

Complement of the relation:

Let $R \subseteq A \times A$ then complement of R is $R' = U - R$.

i.e., $R' = \{(a,b) \in A \times A : (a,b) \notin R\}$

Example: Let the set $A=\{1,2,3\}$ and the relation $R=\{(1,1),(1,2),(1,3),(2,1),(2,3)\}$.

$R' = \{(2, 2), (3, 1), (3, 2), (3, 3)\}$

Equivalence relation

A relation on a set a is called equivalence relation if it is reflexive, symmetric, and transitive.

Example: Let $A = \{1, 2, 3, 4\}$ and $R = \{(1, 1), (1, 3), (2, 2), (2, 4), (3, 1), (3, 3), (4, 2), (4, 4)\}$.

Show that R is an Equivalence Relation.

Solution:

Reflexive: Relation R is reflexive as $(1, 1), (2, 2), (3, 3)$ and $(4, 4) \in R$.

Symmetric: Relation R is symmetric because whenever $(a, b) \in R$, (b, a) also belongs to R .

$(1,3) \in R \Rightarrow (3, 1) \in R$.

$(2, 4) \in R \Rightarrow (4, 2) \in R$.

Transitive: Relation R is transitive because whenever (a, b) and (b, c) belongs to R , (a, c) also belongs to R .

$(3, 1) \in R$ and $(1, 3) \in R \Rightarrow (3, 3) \in R$.

$(2, 4) \in R$ and $(4, 2) \in R \Rightarrow (2, 2) \in R$.

So, as R is reflexive, symmetric and transitive, hence, R is an Equivalence Relation.

Example :Show that the relation is “congruent” is an equivalence relation on a set T of triangles.

Solution:

Reflexive relation:

Let $(T_1, T_1) \in R$

$\Rightarrow T_1$ is congruent to itself.

$\therefore R$ is reflexive.

Symmetric relation:

Let $(T_1, T_2) \in R$

$\Rightarrow T_1$ is congruent to T_2 .

$\Rightarrow T_2$ is congruent to T_1 .

$\Rightarrow (T_2, T_1) \in R$

$\therefore R$ is symmetric.

Transitive relation:

Let $(T_1, T_2) \in R$ and $(T_2, T_3) \in R$

$\Rightarrow T_1$ is congruent to T_2 .

$\Rightarrow T_2$ is congruent to T_3 .

$\Rightarrow T_1$ is congruent to T_3 .

$\Rightarrow (T_1, T_3) \in R$

$\therefore R$ is transitive.

R is reflexive, symmetric and transitive hence it is equivalence relation.

Example : Show that the relation is “set of parallel lines” to is an equivalence relation on a set L of lines.

Solution:

Reflexive relation:

Let $(L_1, L_1) \in R$

$\Rightarrow L_1$ is parallel to itself.

$\therefore R$ is reflexive.

Symmetric relation:

Let $(L_1, L_2) \in R$

$\Rightarrow L_1$ is parallel to L_2 .

$\Rightarrow L_2$ is parallel to L_1 .

$\Rightarrow (L_2, L_1) \in R$

$\therefore R$ is symmetric.

Transitive relation:

Let $(L_1, L_2) \in R$ and $(L_2, L_3) \in R$

$\Rightarrow L_1$ is parallel to L_2 .

$\Rightarrow L_2$ is parallel to L_3 .

$\Rightarrow L_1$ is parallel to L_3 .

$\Rightarrow (L_1, L_3) \in R$

$\therefore R$ is transitive.

R is reflexive, symmetric and transitive hence it is equivalence relation.

Composition of relation

Let R be a relation from a set A to a set B and S a relation from B to a set C . The composite of R and S is the relation consisting of ordered pairs (a, c) , where $a \in A$, $c \in C$ and for which there exists an element $b \in B$ such that $(a, b) \in R$ and $(b, c) \in S$.

We denote the composite of R and S by RoS .

Operation on Relation:

Union of two relations

Given two relations R and S , we will define the Union of these relations, denoted by $R \cup S$,

$R \cup S = \{(a, b) / (a, b) \in R \text{ or } (a, b) \in S\}$

Intersection of two relations

Given two relations R , and S we will define the intersection of these relations, denoted by $R \cap S$

$R \cap S = \{(a, b) / (a, b) \in R \text{ and } (a, b) \in S\}$

Definition Difference of two relations

Given two relations R and S we will define the difference of relations R and S (in that order), denoted by R-S as follows:

$$R - S = \{(a, b) / (a, b) \in R \text{ and } (a, b) \notin S\}$$

$$S - R = \{(a, b) / (a, b) \in S \text{ and } (a, b) \notin R\}$$

Closures operation on relation:**Reflexive closure :**

The **reflexive closure** of a relation R on a set A is the smallest reflexive relation on A that contains R. A relation is called *reflexive* if it relates every element of A to itself.

The reflexive closure R_r of a relation R on a set A is given by

$$R_r = R \cup \{(a, a) / a \in A\}$$

Symmetric closure :

The **symmetric closure** of a relation R on a set A is the smallest symmetric relation on A that contains R.

The symmetric closure R_s of a relation R on a set A is given by

$$R_s = R \cup \{(a, b) / (b, a) \in R \text{ for all } a, b \in A\}$$

Transitive closure:

The **transitive closure** of a relation R on a set A is the smallest transitive relation on A that contains R.

The transitive closure R_t of a relation R on a set A is given by

$$R_t = R \cup \{(a, c) / (a, b) \in R \text{ and } (b, c) \in R \text{ for all } a, b, c \in A\}$$

Partition and Covering:

Let S be a given set and a $\{A_1, A_2, A_3, \dots, A_m\}$ where each A_i is a subset of S and $\bigcup_{i=1}^m A_i = S$

Then the set A is called a covering of the sets, $A_1, A_2, A_3, \dots, A_m$ are said to cover S.

The elements of A, which are subsets of S, are mutually disjoint then a is called partition of S, and the sets $A_1, A_2, A_3, \dots, A_m$ are called the blocks of the partition.

Representation of relation:**Relation matrix:**

A relation R from a finite set A to B can be represented by a Zero one matrices called relation matrix of R.

$$M_R = [m_{ij}] = \begin{cases} 1 & \text{if } (a, b) \in R \\ 0 & \text{if } (a, b) \notin R \end{cases}$$

Union, Intersection of relation matrix:

The Boolean operations join and meet can be used to find the matrices representing the union and intersection of two relations. Suppose that R and S are relations on a set A represented by the matrices M_R and M_S , respectively.

That matrix representing the union of these relations has a 1 in the positions where either M_R or M_S has a 1. The matrix representing the intersection of these relations has a 1 in the where both M_R and M_S have a 1.

Thus, the matrices representing the union and intersection of these relations are

$$M_{R \cup S} = M_R \vee M_S$$

$$M_{R \cap S} = M_R \wedge M_S$$

$$M_{R \oplus S} = M_R \oplus M_S$$

Representing Relations Using Digraphs

Each element of the set is represented by a point, and each ordered pair is represented using an arc with its direction indicated by an arrow. We use such pictorial representations when we think of relations on a finite set as directed graphs, or digraphs.